COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010	**SECTION:**	CPE21S2
DATE PERFORMED:	07/10/26	**DATE SUBMITTED:**	7/12/26
NAME:	SUBALISID, JAPPETH ROYCE B.	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. HOA 1.1 Basic C++ Programming

1. PROCEDURE OUTPUT

ILO A: Create Code That Follows the Basic C++ Code Structure

For this activity, you have to demonstrate the use of a function prototype. The section on class declaration and method definition will be used for the function prototype and the function will be defined in the follow method definition section after the main function.

A function prototype in c++ is a declaration of the name, parameters and return type of the function before its definition. Write a C++ code the satisfies the following:

- Create a function that will take two numbers and display the sum.

```
int sumOfTwo(int x, int y) {
    int sum = x + y;
    return sum;
};
```

```
//sum function call
int x = 3, y = 4;
std::cout << x << " + " << y << " = " << sumOfTwo(x,y) << '\n';
```

Output: `3 + 4 = 7`

- Create a function that will return whether variable A is greater than variable B.

```
bool isGreater(float a, float b) {
    if(a > b) {return true;
    }else {return false;}
};
```

```
//comparing numbers function call
int A = 67;
int B = 69;
if(isGreater(A, B)) {
    std::cout << A << " is greater than " << B << '\n';
}
else {
    std::cout << A << " is not greater than " << B << '\n';
}
```

Output: `67 is not greater than 69`

- Create a function that will take two Boolean values and display the result of all logical operations then return true if it was a success.

```cpp
template <typename T1, typename T2>
bool boolOperations(T1 a, T2 b) {
    if(std::is_same<T1, bool>::value && std::is_same<T2, bool>::value) {
        if(a && b) {std::cout << "AND = True\n"; //AND operation
        }else {std::cout << "AND = False\n";}


        if(a || b) {std::cout << "OR = True\n"; //OR operation
        }else {std::cout << "OR = False\n";}


        if(!(a && b)) {std::cout << "NAND = True\n"; //NAND operation
        }else {std::cout << "NAND = False\n";}


        if(!(a || b)) {std::cout << "NOR = True\n"; //NOR operation
        }else {std::cout << "NOR = False\n";}


        if(a != b) {std::cout << "XOR = True\n"; //XOR operation
        }else {std::cout << "XOR = False\n";}


        return true;
    } else {
        return false;
    }
}
```

```cpp
//bool operation function call
auto argA = 12;
auto argB = false;
if(boolOperations(argA, argB)) {
    std::cout << "SUCCESS!\n";
}else {
    std::cout << "FAIL!\n";
}
```

```
FAIL!
```

```cpp
//bool operation function call
auto argA = true;
auto argB = false;
if(boolOperations(argA, argB)) {
    std::cout << "SUCCESS!\n";
}else {
    std::cout << "FAIL!\n";
}
```

```
AND = False
OR = True
NAND = True
NOR = False
XOR = True
SUCCESS!
```

Note:

- The driver program must call each function.
- The definitions must be after the main function.

```
#include <iostream>
#include <type_traits>


int sumOfTwo(int x, int y);
bool isGreater(float a, float b);
template <typename T1, typename T2>
bool boolOperations(T1 a, T2 b);

class Triangle{···

int main() {···


int sumOfTwo(int x, int y) {···


bool isGreater(float a, float b) {···

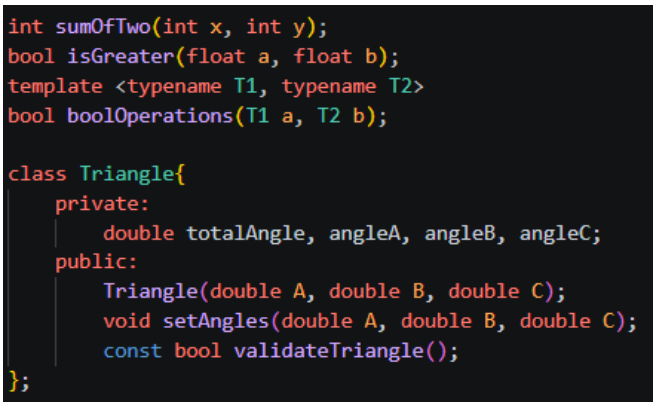
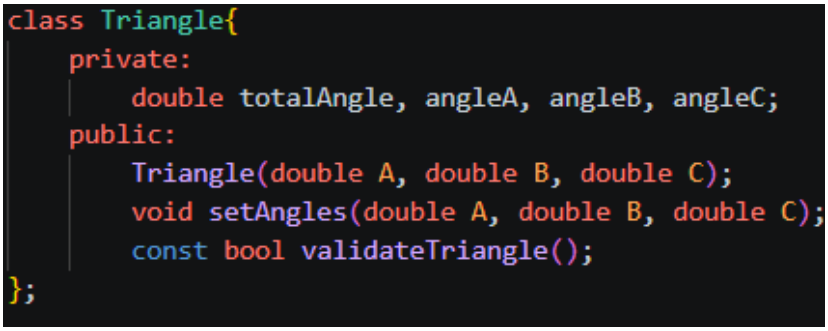
template <typename T1, typename T2>
bool boolOperations(T1 a, T2 b) {···
```

ILO B: Implement Appropriate Class Definition and Instances Based on Given Requirements

In this section, the initial implementation for a class triangle will be implemented. The step-by-step procedure is shown below:

- Step 1. Include the necessary header files. For this one, we only need #include <iostream>
- Step 2. Create the triangle class. Assign it with private variables: totalAngle, angleA, angleB, and angleC.
- Step 3. We then create public methods. The constructor must allow for creation of the object with 3 initial angles to be stored in our previously defined variables angleA, angleB and angleC. Another method has to be made if the user wants to change the initial values, this will also accept 3 arguments to change the values in angleA, angleB and angleC. Lastly, a function to validate whether the given values make our shape an actual triangle.
- Step 4. Define the methods.
- Step 5. Create the driver code.

Sections	Answer
Header File Declaration Section	`#include <iostream>` `#include <type_traits>`

Global Declaration Section	
Class Declaration and Method Definition Section	

Main Function

```cpp
int main() {
    //sum function call
    int x = 3, y = 4;
    std::cout << x << " + " << y << " = " << sumOfTwo(x,y) << '\n';

    //comparing numbers function call
    int A = 67;
    int B = 69;
    if(isGreater(A, B)) {
        std::cout << A << " is greater than " << B << '\n';
    }
    else {
        std::cout << A << " is not greater than " << B << '\n';
    }

    //bool operation function call
    auto argA = true;
    auto argB = false;
    if(boolOperations(argA, argB)) {
        std::cout << "SUCCESS!\n";
    }else {
        std::cout << "FAIL!\n";
    }

    //ILO 2 Function call
    int angleA, angleB, angleC;
    std::cout << "Enter the three angles of a triangle: ";
    std::cin >> angleA >> angleB >> angleC;
    Triangle tri1(angleA, angleB, angleC);
    if(tri1.validateTriangle()) {
        std::cout << "The triangle is valid.\n";
    }else {
        std::cout << "The triangle is not valid.\n";
    }
    return 0;
}
```

Method Definition

```cpp
Triangle::Triangle(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;
};
```

```cpp
void Triangle::setAngles(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;
};
```

	`const bool Triangle::validateTriangle(){`
	`  return (totalAngle == 180);`
	`};`

Table 1-1. C++ Structure Code for Answer

```
3 + 4 = 7
67 is not greater than 69
AND = False
OR = True
NAND = True
NOR = False
XOR = True
SUCCESS!
Enter the three angles of a triangle: 15
45
90
The triangle is not valid.
```

Table 1-2. ILO B output observations and comments.

The first two functions in ILO A are pretty self explanatory and easy to do since we've already done them many times. The boolean operation, however, was difficult since unlike python there is no built-in 'isBool()' function in c++ and I have never tried doing it so I had to outsource and learn about templates. It was pretty complicated since it also uses a function in the standard library.

The ILO B in the lab already provided the code to the answer and we just had to follow it, I also added an angle input for the user to get angles without hardcoding it—I should've done the same with the ILO A.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve Different Problems using the C++ Programming Language

The supplementary activities are meant to gauge your ability in using C++. The problems below range from easy to intermediate to advanced problems. Note your difficulties after answering the problems below.

1. Create a C++ program to swap the two numbers in different variables.

```cpp
#include <iostream>

int swapNumbers(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
    return 0;
}

int main() {
    // Swap numbers function call
    int num1, num2;
    std::cout << "Enter two numbers:\n";
    std::cin >> num1 >> num2;
    swapNumbers(num1, num2);
    std::cout << "After swapping:\n" << num1 << "\n" << num2 << '\n';

    return 0;
}
```

```
Enter two numbers:
6
7
After swapping:
7
6
```

2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.

```cpp
// Kelvin to Fahrenheit function call
double kelvin;
std::cout << "Enter temperature in Kelvin: ";
std::cin >> kelvin;
std::cout << kelvin << " Kelvin is equal to " << kelvinToFahrenheit(kelvin) << " Fahrenheit.\n";
```

```cpp
double kelvinToFahrenheit(double kelvin) {
    double celsius = kelvin - 273.15;
    double fahrenheit = (celsius * 9/5) + 32;
    return fahrenheit;
}
```

```
Enter temperature in Kelvin: 300
300 Kelvin is equal to 80.33 Fahrenheit.
```

3. Create a C++ program that has a function that will calculate the distance between two points.

```cpp
// Distance function call
double x1, y1, x2, y2;
std::cout << "Enter coordinates of first point (x1 y1):\n";
std::cin >> x1 >> y1;
std::cout << "Enter coordinates of second point (x2 y2):\n";
std::cin >> x2 >> y2;
std::cout << "The distance between the two points is: " << distance(x1, y1, x2, y2) << "units. \n";
```

```
double distance(double x1, double y1, double x2, double y2) {
    double dist = sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2));
    return dist;
}
```

```
Enter coordinates of first point (x1 y1):
0
0
Enter coordinates of second point (x2 y2):
16
18
The distance between the two points is: 24.0832units.
```

4. Modify the code given in ILO B and add the following functions:

a. A function to compute for the area of a triangle

```
double sideA;
std::cout << "Enter the length of side opposite of angle A: ";
std::cin >> sideA;
std::cout << "The area of the triangle is: " << tri1.areaOfTriangle(sideA) << " square units.\n";
```

```
double Triangle::areaOfTriangle(double sideA) {
    if (!validateTriangle()) {
        std::cout << "The triangle is not valid. Cannot calculate area.\n";
        return -1;
    }else {
        double area = (pow(sideA, 2) * sin(angleB* M_PI / 180) * sin(angleC * M_PI / 180)) / (2 * sin(angleA * M_PI / 180));
        return area;
    }
};
```

```
Enter the three angles of a triangle: 30
60
90
The triangle is valid.
Enter the length of side opposite of angle A: 30
The area of the triangle is: 779.423 square units.
```

b. A function to compute for the perimeter of a triangle

```
std::cout << "The perimeter of the triangle is: " << tri1.perimeter(sideA) << " units.\n";
```

```
double Triangle::perimeter(double sideA) {
    if (!validateTriangle()) {
        std::cout << "The triangle is not valid. Cannot calculate perimeter.\n";
        return -1;
    }else {
        double sideB = (sideA * sin(angleB * M_PI / 180)) / sin(angleA * M_PI / 180);
        double sideC = (sideA * sin(angleC * M_PI / 180)) / sin(angleA * M_PI / 180);
        double perimeter = sideA + sideB + sideC;
        return perimeter;
    }
};
```

```
Enter the three angles of a triangle: 30
60
90
The triangle is valid.
Enter the length of side opposite of angle A: 30
The area of the triangle is: 779.423 square units.
The perimeter of the triangle is: 141.962 units.
```

c. A function that determines whether the triangle is acute-angled, obtuse-angled or 'others.'

```
std::cout << "The triangle is " << (tri1.isObtuse() ? "obtuse." : "not obtuse.") << '\n';
```

```
const bool Triangle::isObtuse() {
    if (!validateTriangle()) {
        std::cout << "The triangle is not valid. Cannot determine if it is obtuse.\n";
        return false;
    }else {
        return (angleA > 90 || angleB > 90 || angleC > 90);
    }
};
```

```
Enter the three angles of a triangle: 30
60
90
The triangle is valid.
Enter the length of side opposite of angle A: 30
The area of the triangle is: 779.423 square units.
The perimeter of the triangle is: 141.962 units.
The triangle is not obtuse.
```

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Gemini

Purpose of Use:

Learning new concepts, Math formulas, c++ library commands

Extent of Use:

Learned how to do templating, asked for math formulas for the triangle calculations, and cmath functions.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.



I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.